TEMPO

From raw video to retrieval-grounded visual question answering

Research development record | Part 1 of 3 | Stages 1–5

This report reconstructs the first five implementation stages of the multimodal video retrieval-augmented generation (Video RAG) prototype. It explains the intended research goal, algorithms, software components, experiments, observed outputs, and limitations of each stage. Later reports will cover benchmark evaluation, object tracking, identity memories, and uncertainty-aware association.



Figure 1. System-development progression (conceptual; not a measured performance plot).

Scope and evidence standard

The sequence and implementation details are reconstructed from the project work discussed in our conversations. We report numerical results only where an actual run was recorded. Some early stages were verified with smoke tests rather than standardized accuracy benchmarks. Illustrations in this report are explicitly marked as conceptual or derived from recorded observations; no unrecorded accuracy, latency, or hardware-throughput values are invented.

Stage map

Stage	Objective	Key deliverable
1	Turn video into timestamped visual samples	Frame extractor
2	Align frames and language in one embedding space	CLIP encoder
3	Find relevant frames efficiently	FAISS retrieval index
4	Answer a visual question using retrieved evidence	Single-frame Qwen2.5-VL pipeline
5	Include before/after context for event questions	Multi-frame temporal retrieval and VLM reasoning

Important distinction: these stages establish an end-to-end prototype, not yet a validated long-video reasoning method. The systematic YouCook2 retrieval evaluation and BURST identity experiments belong to subsequent stages.

Stage 1 | Video preprocessing and timestamped frames

1.1 What we wanted to achieve

A video-language model cannot efficiently reason over an arbitrarily long raw video as one undifferentiated input on a local GPU. The first goal was to convert a video into a manageable, time-indexed collection of frames. Each frame needed a stable link to its original moment so retrieval results could be interpreted and eventually cited.

1.2 Why frame sampling is necessary

For a video of duration T seconds, sampling every Δt seconds yields approximately N ≈ T/Δt frames (subject to endpoints and decoding). Sampling reduces computation and storage but discards motion between sampled frames. This creates an explicit trade-off: coarse sampling is fast but may miss short actions; dense sampling captures more events but increases embedding and retrieval cost.

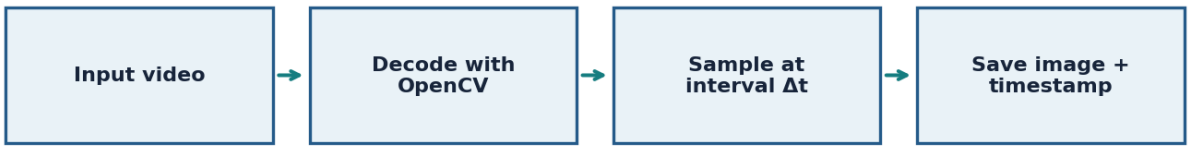


Figure 2. Stage 1 processing pipeline (conceptual).

1.3 What we implemented

The project introduced src/perception/video_processor.py with a VideoProcessor interface. The working configuration used a sampling interval of 1.0 second and a maximum frame resolution of 640 pixels. OpenCV decoded the video, selected frames, and saved the image paths and associated timestamps for downstream processing.

A representative constructor from the working implementation was VideoProcessor(video_path, output_dir, sampling_interval=1.0, max_resolution=640), followed by extract_frames(). The exact count of extracted frames varies with source duration, sampling interval, and endpoint handling.

1.4 What we tested and observed

Frame extraction successfully produced usable image samples and timestamp metadata for subsequent CLIP and VLM experiments. A later 332.5-second YouCook2 example produced 167 frames when sampled at approximately two-second intervals; that later run is useful as an illustration of sampling density, but it was not the Stage 1 benchmark.

1.5 Result and limitation

Result: We established a reproducible video-to-timestamped-frames interface. Limitation: Individual frames do not encode motion trajectories or causal event order. Sampling can omit the decisive instant of a short action. These limitations motivated both retrieval and later temporal-context expansion.

Questions to keep in mind

- How much temporal information is lost at a one- or two-second sampling interval?
- Should frame sampling adapt to action density or motion?
- How should timestamp accuracy be preserved after resizing and batching?

Stage 2 | CLIP image and text embeddings

2.1 What we wanted to achieve

We needed a way to retrieve frames using natural-language queries without first generating captions for every frame. CLIP provides an image encoder and a text encoder trained to place semantically related images and descriptions near one another in a shared feature space.

2.2 The underlying mechanism

Let f(I) be the image embedding and g(q) the query embedding. We normalize each vector to unit length and compute cosine similarity:

similarity(I, q) = [f(I) · g(q)] / [||f(I)|| ||g(q)||].

After L2 normalization, cosine similarity is simply the dot product. A high score indicates semantic alignment under CLIP, not a guarantee that the queried event actually occurs in that frame. CLIP can confuse visually similar actions or objects and does not inherently model before/after relationships.



Figure 3. Shared-embedding retrieval idea (conceptual; the text query is encoded by the separate CLIP text tower).

2.3 What we implemented

The implementation used src/embeddings/clip_encoder.py, a CLIPEncoder class, and the pretrained openai/clip-vit-base-patch32 model. It supported encode_images(image_paths, batch_size=16) and encode_text([query]). The output was a normalized 512-dimensional embedding per image or text query.

2.4 What we tested and observed

The initial smoke test encoded nine frames and three text queries, yielding image embeddings with shape 9 × 512 and text embeddings with shape 3 × 512. Query rankings returned relevant frames in the small test set. This established functional semantic alignment and correct tensor dimensions, not population-level retrieval accuracy.

2.5 Result and limitation

Result: We could compare language queries directly with video frames numerically. Limitation: A single-frame embedding is a weak representation of temporally extended events; it may recognize a ladder or flag without knowing what happened before or after an action.

Stage 3 | FAISS similarity search

3.1 What we wanted to achieve

Once every sampled frame has an embedding, a query must search those embeddings and return the best-matching timestamps. The first prototype could use NumPy dot products, but we also wanted an index that could support a larger frame collection and be saved or reloaded independently of embedding computation.

3.2 How retrieval works

Given a matrix E of normalized frame embeddings and a normalized query vector q, the retrieval scores are s = E q. The highest-scoring indices correspond to the most semantically similar frames. We used FAISS IndexFlatIP, which performs exact inner-product search; for normalized vectors, this ranks frames by cosine similarity. Exact search is simple and deterministic but is not an approximate-nearest-neighbor speedup.



Figure 4. Exact vector-search workflow (conceptual).

3.3 What we implemented

The component src/retrieval/faiss_retriever.py provided FAISSRetriever with methods to build_index(embeddings, metadata), search(...), save(...), and load(...). Metadata tied each embedding index back to its frame and timestamp. Separating the index from the VLM meant that the expensive reasoning model would only receive a small subset of video evidence.

3.4 What we tested and observed

On the nine-frame test, FAISS returned the same top-ranked indices as a direct NumPy similarity computation. One recorded ranking was [7, 3, 8, 6, 0]. The agreement verified that vector normalization, indexing, and retrieval ordering were consistent for that example.

3.5 Result and limitation

Result: We had a working text-to-frame retrieval engine with persistent index support. Limitation: Ranking individual frames does not ensure event coverage, temporal diversity, or an answerable set of evidence. Closely spaced frames can dominate the top-k, while an important earlier or later frame may be omitted.

Why this matters for the research

The FAISS stage established a reproducible baseline that could later be evaluated using annotated event intervals. It also made it possible to distinguish retrieval failure from reasoning failure: if the correct moment is never retrieved, a downstream VLM cannot reliably answer using that evidence.

Stage 4 | Single-frame retrieval-augmented VLM reasoning

4.1 What we wanted to achieve

The retrieval engine identifies candidate visual evidence but does not itself produce a natural-language answer. We therefore connected it to a vision-language model capable of interpreting an image and responding to a question. The goal was to build the first complete query → retrieval → answer path.

4.2 Architecture

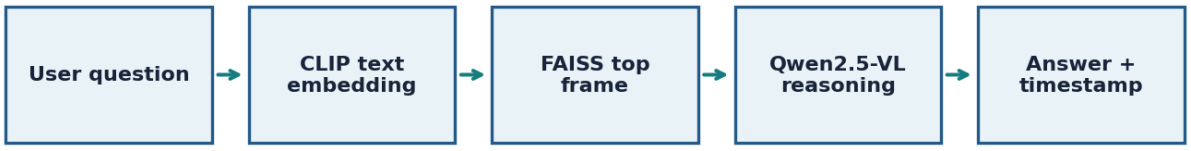


Figure 5. Single-frame retrieval-augmented question answering (conceptual).

4.3 What we implemented

The reasoning module src/reasoning/vlm.py used Qwen/Qwen2.5-VL-3B-Instruct on the local CUDA GPU. The stable configuration used bfloat16 (BF16) with SDPA attention and device_map="auto". An earlier FP16 configuration produced repetitive punctuation in generated text; switching to BF16 resolved that observed behavior. One successful run reported approximately 7.56 GB of allocated GPU memory for the model.

4.4 What we tested and observed

A standalone VLM test successfully described an image, confirming that the processor, model weights, image input, and text generation path worked. We then asked “What is the person holding?” in the single-frame RAG pipeline. The retriever selected a frame at 6.00 seconds, with recorded CLIP similarity 0.2325, and supplied that image to the VLM.

The 0.2325 value is an uncalibrated similarity score, not a 23.25% confidence or accuracy estimate. The available record confirms that retrieval selected a timestamped frame; it does not preserve a standardized ground-truth correctness assessment of the final answer.

4.5 Result and limitation

Result: The full local pipeline ran end-to-end: a natural-language question selected a relevant frame, and a VLM processed that evidence. Limitation: A single frame is often insufficient for questions containing “after,” “before,” “then,” or “why.” The image may depict an intermediate state without showing the sequence of actions.

Stage 5 | Multi-frame retrieval and temporal context

5.1 What we wanted to achieve

We wanted to answer questions about the evolution of an event rather than the contents of one isolated image. The key idea was to retrieve relevant frames and expand them with temporal neighbors so that the VLM could inspect a short sequence with known timestamps.

5.2 Why a single best frame is insufficient

Consider the question “What happens after the person climbs the ladder?” A frame showing the person on the ladder may be relevant to the words in the question, yet it does not establish the subsequent action. The answer requires visual evidence from later moments. Temporal expansion is a lightweight way to recover some of this context without encoding every frame jointly.



Figure 6. Multi-frame retrieval-augmented reasoning (conceptual).

5.3 What we implemented

We added src/retrieval/temporal_retriever.py to expand retrieval around relevant timestamps and passed multiple frames to Qwen2.5-VL. Rather than treating each retrieved image independently, the reasoning prompt could use ordered evidence to describe the sequence of events. The same CLIP encoder and FAISS index from Stages 2 and 3 were reused.

5.4 What we tested and observed

For the ladder question, the recorded retrieval returned frames at 10.00 s, 12.00 s, and 14.00 s. On visual inspection, the person remained near the top of the ladder and attempted to hang a flag. This demonstrated why context matters: the correct interpretation is not simply that the person finished climbing, but that they continued performing a task afterward.

An early multi-frame test was also reported to produce a coherent temporal response using four retrieved frames. The preserved records do not include an independently scored answer or a full table of temporal question-answer results. Consequently, this stage should be treated as a qualitative proof of concept rather than evidence of measured temporal-reasoning accuracy.

5.5 Result and limitation

Result: The pipeline could retrieve multiple timestamped frames and use them together for event-oriented question answering. Limitation: Sampling, retrieval, and neighborhood expansion are heuristic. The system still lacks explicit object identity, persistent object memory, long-range temporal linking, and a systematic evaluation protocol.

Synthesis | What the first five stages established

The implemented system



Figure 7. The working baseline after Stage 5 (conceptual system diagram).

Component	Concrete implementation	Verification
Frame processing	OpenCV; VideoProcessor; timestamped frame images	Successful frame extraction

Component	Concrete implementation	Verification
Image/text representation	CLIP ViT-B/32; normalized 512-D vectors	9 image embeddings, 3 query embeddings
Search	FAISS IndexFlatIP + metadata	Matches NumPy top-k ordering
Visual reasoning	Qwen2.5-VL-3B, CUDA, BF16	Successful image description and single-frame QA path
Temporal context	Temporal retrieval/neighbor expansion	Ladder/flag sequence inspected across timestamps

What we can and cannot claim

- Supported: An operational local Video RAG prototype that accepts a question, retrieves timestamped visual evidence, and invokes a VLM for an answer.
- Supported: Basic retrieval correctness on a small smoke test and qualitative usefulness of multi-frame context.
- Not yet supported: A numerical retrieval recall, temporal QA accuracy, state-of-the-art comparison, or proof of causal reasoning for Stages 1–5.
- Not yet supported: Robust object identity across occlusion, reappearance, or visually similar instances.

Why we needed the next stages

The prototype demonstrated feasibility but did not tell us how often retrieval found the correct moments. It also had no way to preserve object identity across time. The next development phase therefore introduced real annotated videos, retrieval metrics, and error analysis. That evaluation exposed weaknesses of frame-level semantics and motivated the object-centric memory research that later evolved into TEMPO.

Research takeaway

The first five stages are best understood as a controlled engineering foundation. They isolate the major components of a video-language retrieval system so later research can measure which component fails and whether a proposed improvement fixes that failure. This separation is essential for a publishable experimental methodology.

Technical appendix | Reproducibility and terminology

Software modules recorded in the project

Path	Responsibility
src/perception/video_processor.py	Video decoding, sampling, timestamps, resizing
src/embeddings/clip_encoder.py	CLIP image and text embeddings
src/retrieval/faiss_retriever.py	Exact similarity index and metadata
src/retrieval/temporal_retriever.py	Temporal-context expansion
src/reasoning/vlm.py	Qwen2.5-VL image/video reasoning

Definitions

- Frame sampling: Choosing still images at selected timestamps rather than processing every video frame.
- Embedding: A learned vector representation used for numerical similarity comparisons.
- Cosine similarity: Normalized dot product; a retrieval score, not a calibrated probability.
- Top-k retrieval: Selecting the k highest-ranked candidates under a similarity function.
- RAG: Retrieval-augmented generation; using retrieved evidence as input to a generative model.
- Temporal grounding: Connecting a claim or answer to when its supporting evidence appears in a video.

Chronology and provenance notes

This report documents the first five stages of the CLIP/FAISS/Qwen2.5-VL Video RAG development path discussed in the project. Some other exploratory vision-language work may have used different models or datasets; those experiments are not silently merged into these five stages. Numeric facts are restricted to the recorded tests, and unmeasured claims are identified as limitations or hypotheses.

The next volume will begin with the transition from prototype demonstrations to systematic retrieval evaluation and object-centric experiments. It will preserve the same structure: objective, method, implementation, observed results, interpretation, and limitations.